[Khanh Pham] Threads & Synchronization

Thread

After understanding processes, the next step is to look at threads. A thread is the smallest unit of execution within a process. While a process provides an isolated execution environment, threads within the same process share resources such as memory and file descriptors.

This section will focus on an overview of threads, synchronization, and synchronization mechanisms like mutexes, condition variables, read-write locks, barier, spinlock and semaphores. I will also discuss issues that arise from thread synchronization, such as deadlocks, starvation and livelocks.

> I will not cover basic knowledge that can easily be found in Linux books, such as the definition of thread functions, their arguments, or their attributes.

Thread overview

The reason why we need thread

A thread is the smallest unit of execution within a program, and each process can have more than one thread.

> So why do we need threads? Why not just create multiple processes instead?

- Each process has its own separate memory space. However, some tasks do not need a separate memory space; they may only need to perform a simple logic task. In this case,

creating a new process can introduce unnecessary overhead.
- Processes do not share memory by default, so if two processes need to communicate with each other, we need to use IPC. Threads solve this problem by allowing multiple threads within the same process to share the same memory space while performing different tasks.
- When we use multithread programming, we can use more power of hardware(CPU)

When I compare the different of running time between **single thread** and **multithread**:

```
N = 20000000

Single-thread
Result = 59648481635.85
Time   = 1.172324 seconds

2 threads
Result = 59648481635.85
Time   = 0.617080 seconds
Speedup = 1.90x

4 threads
Result = 59648481635.85
Time   = 0.407601 seconds
Speedup = 2.88x

8 threads
Result = 59648481635.85
Time   = 0.375024 seconds
Speedup = 3.13x
```

As we can see, the execute time when we use multi thread is shorter than single thread. A source of example here:

https://github.com/220222112-PhamNgocKhanh/Zhone_trainee/blob/main/thread_and_Synchronization/demo_single_multi_thread.c

What about when we want to interact with thread?

Thread ID, creation and termination

As we know, every processes has a process ID, every threads has a thread ID too. Unlike the process ID, which is unique in the system, the thread ID has significance only within the context of the process to which it belongs

When we create more threads, like process, we can't know which thread will be run first. An example:

```
void *threadA(void *arg)
{
    for (int i = 0; i < 100; i++)
    {
        printf("A\n");
    }

}



int main()
{
    pthread_t a;

    pthread_create(&a, NULL, threadA, NULL);

    for (int i = 0; i < 100; i++)
    {
        printf("main\n");
    }

    return 0;
}
```

The result sometime look like bellow. We can see, the main thread and threadA run together in the same time.

```
A
A
main
main
main
A
A
```

```
A
A
```

Now if we count the number of `A` lines in the output, sometimes we may notice that there are **fewer than 100 lines of** `A`. Why does this happen?

The reason in the end of the `main` function:

```
    pthread_t a;

    pthread_create(&a, NULL, threadA, NULL);

    for (int i = 0; i < 100; i++)
    {
        printf("main\n");
    }

    return 0;
```

When the main thread done, the process also is died, so every threads that are still running will be terminated too.

> **What we can do in this situation?**

As we know, If any thread within a process calls `exit`, `_Exit`, or `_exit`, then the entire process terminates.

Thread can exit by 3 ways:

- The thread can simply return from the start routine. The return value is the thread's exit code.
- The thread can be canceled by another thread in the same process.
- The thread can call `pthread_exit`.

In the example above, if we call `pthread_exit` to termninate the main thread without killing the process. So both `main` and `threadA` can be completed.

Now we know how we can make the thread terminate. If we want a thread must wait another thread, we can use the `pthread_join` to wait other thread by passing the tid of this thread. We can imagine this function like the `wait` function in process.

An example:

```
struct temp
{
    int a;
    int b;
};

void *thread_cal(void *arg)
{
    struct temp *t = (struct temp *)arg;

    int *result = malloc(sizeof(int));
    *result = t->a + t->b;
    sleep(2);
    return result;
}

void *thread_print(void *arg)
{
    pthread_t tid = *(pthread_t *)arg;

    int *ptr;
    pthread_join(tid, (void*) &ptr);
    int result = *ptr;
    printf("The result is: %d\n", result);
    free(ptr);
    return NULL;
}

int main()
{
    pthread_t tid1, tid2;

    struct temp t;

    t.a = 5;
    t.b = 6;

    pthread_create(&tid1, NULL, thread_cal, &t);
```

```
    pthread_create(&tid2, NULL, thread_print, &tid1);
    pthread_join(tid2, NULL);

    return 0;
}
```

The result:

```
The result is: 11
```

Because we use the `print` thread wait the `cal` thread, then the `main` thread wait the `print` thread, so the program is nomally running.

As we mentioned earlier, one thread can be terminated by another thread. We can do this using the `pthread_cancel` function.

We can use the `pthread_detach` function to be reclaimed automatically once the thread terminates. However, if we use `pthread_detach`, we can't use `pthread_join` for this thread. For example:

```
int count = 0;
void *thread1(void *arg)
{
    for (int i = 0; i < 100000; i++)
    {
        count++;
    }
    pthread_t pid = pthread_self();

    pthread_exit(NULL);
}

int main()
{
    pthread_t tid;

    pthread_create(&tid, NULL, thread1, NULL);
    pthread_detach(tid);
    pthread_join(tid, NULL);

    printf("%d", count);
```

```
    return 0;
}
```

In this example, although the `main` thread have `pthread_join`, the `main` thread sometimes terminate and make the process terminate before the `thread1` done.

```
./test
0
./test
464
```

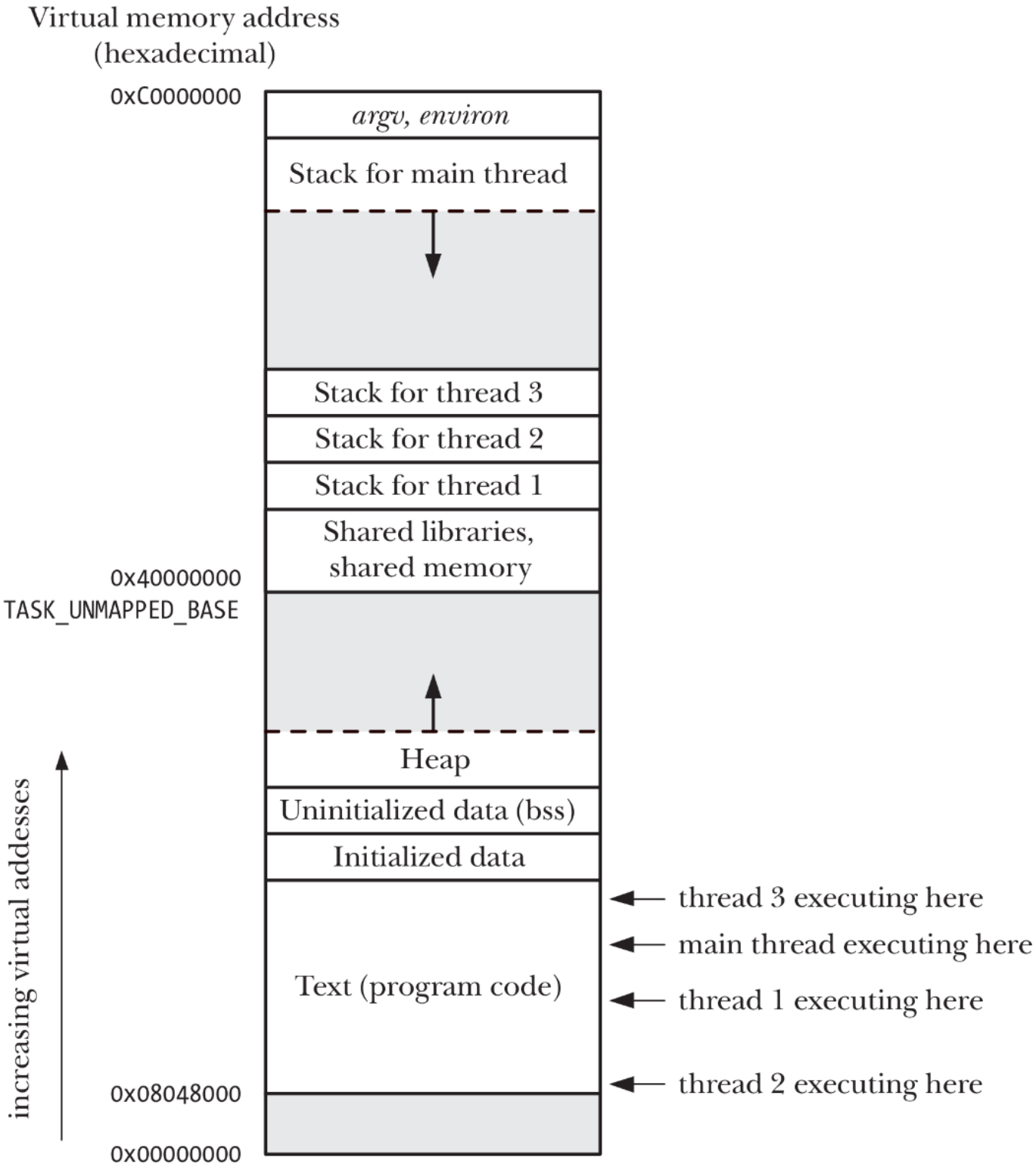
This table the relationship between the process function and thread function:

Process primitive	Thread primitive	Description
`fork`	`pthread_create`	create a new flow of control
`exit`	`pthread_exit`	exit from an existing flow of control
`waitpid`	`pthread_join`	get exit status from a flow of control
`atexit`	`pthread_cleanup_push`	register a function to be called at exit from a flow of control
`getpid`	`pthread_self`	get ID for a flow of control
`abort`	`pthread_cancel`	request abnormal termination of a flow of control

After understanding the thread overview, we now need to understand how threads share memory with others.

Thread Synchronization

As we know, each process have and unique memory layout and each process also have at least 1 thread. Therefore, each thread data will stored in unique stack, but they will shared all data in other areas like: text, data,bss and heap.



What about if more than 1 thread change shared data at the same time:

For example:

```
int count = 0;
void *thread1(void *arg)
{
    for (int i = 0; i < 100000; i++)
```

```
    {
        count++;
    }
}

int main()
{
    pthread_t tid;

    pthread_create(&tid, NULL, thread1, NULL);
    for (int i = 0; i < 100000; i++)
    {
        count++;
    }
    pthread_join(tid, NULL);

    printf("%d", count);

    return 0;
}
```

The result when I run 3 times:

```
125427
111300
112197
```

As we can see, the result is not `200000` as we expected. Why does this happen?

When the program runs, there are moments when both the `main` and `thread1` try to increment `count` at the same time. Although there are two increment operations, the value of `count` may increase by only `1` because both threads can read the same value before either of them writes the updated value back. This is a **race condition**.

So we know, the race condition is the situation that more than 1 thread interact with the shared data, so we can't control the output of each thread. We can fix this problem by some ways that I will write in the next section:

Mutexes

A mutex is basically a lock that we acquire (`lock`) before accessing a shared resource and release (`unlock`) once we're done.

However, it's important to understand that a mutex does not protect the data by itself. Instead, we must design our program logic around the mutex's state.

Let's fix the example above by using a mutex:

```
pthread_mutex_t mutex;

int count = 0;
void *thread1(void *arg)
{
    pthread_mutex_lock(&mutex);
    for (int i = 0; i < 100000; i++)
    {
        count++;
    }
    pthread_mutex_unlock(&mutex);
}

int main()
{
    pthread_t tid;
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&tid, NULL, thread1, NULL);
    pthread_mutex_lock(&mutex);
    for (int i = 0; i < 100000; i++)
    {
        count++;
    }
    pthread_mutex_unlock(&mutex);
    pthread_join(tid, NULL);

    printf("%d", count);

    return 0;
}
```

So now the result is always `200000`.

> What happen if one thread doesn't unlock mutex or double lock mutex?

- If a thread locks a mutex and never unlocks it, while other threads are waiting for the same mutex until the lock is released, **this thread will run forever.**
- If a thread locks a mutex and then tries to lock the same mutex again, the program will also run forever. This happens because the thread must wait for the mutex that it is already holding.

> We can use the `pthread_mutex_trylock` function to check whether the mutex is locked. If it is locked, the function returns, allowing us to do other work. Otherwise, it locks the mutex, and we can perform the logic with the shared data. We can also use `pthread_mutex_timedlock` to wait for the mutex for a specific amount of time.

Read-writer locks

`Read-writer locks` are the advanced mutexes, they have 3 states: **locked in read mode**, **locked in write mode**, and **unlocked**.

Only one thread at a time can hold a `reader–writer lock` in write mode, but multiple threads can hold a `reader–writer lock` in read mode at the same time.

Look an example:

```
pthread_rwlock_t rwlock;
int number = 0;

void *read1(void *arg)
{
    while (1)
    {
        pthread_rwlock_rdlock(&rwlock);
        printf("read1: %d\n", number);
        sleep(1);
        pthread_rwlock_unlock(&rwlock);
    }
}

void *read2(void *arg)
```

```
{
    while (1)
    {
        pthread_rwlock_rdlock(&rwlock);
        printf("read2: %d\n", number);
        sleep(1);
        pthread_rwlock_unlock(&rwlock);
    }
}

void *write1(void *arg)
{
    while (1)
    {
        pthread_rwlock_wrlock(&rwlock);
        printf("write1\n");
        number++;
        pthread_rwlock_unlock(&rwlock);
    }
}

void *write2(void *arg)
{
    while (1)
    {
        pthread_rwlock_wrlock(&rwlock);
        printf("write2\n");
        number++;
        pthread_rwlock_unlock(&rwlock);
    }
}

int main()
{
    pthread_t tid1, tid2, tid3, tid4;
    pthread_rwlock_init(&rwlock, NULL);
    pthread_create(&tid1, NULL, read1, NULL);
    pthread_create(&tid2, NULL, write1, NULL);
    pthread_create(&tid3, NULL, write2, NULL);
    pthread_create(&tid4, NULL, read2, NULL);
```

```
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
    pthread_join(tid3, NULL);
    pthread_join(tid4, NULL);

    pthread_rwlock_destroy(&rwlock);
}
```

The result:

```
read2: 0
read2: 0
read1: 0
write1
read1: 1
read2: 1
write1
read2: 2
read1: 2
read1: 2
read2: 2
write1
read2: 3
read1: 3
read2: 3
```

> There is only 1 type of lock at the same time. If a thread lock the write lock, other thread must wait until the lock of write mode release

Condition variable

Condition variable is the technique that we keep threads sleep until receive a signal to wake up.

> The function `pthread_cond_wait` will **atomic unlock the mutex before sleeping**. This function will also **atomic lock the mutex after waking up** too.

The standard implement when we use the `pthread_cond_wait` is:

```
while(condition check)
{
    pthread_cond_wait();
}
```

So why we must use `while` in this situation, what about if we use `if` instead?

```
pthread_mutex_t mutex;
pthread_cond_t cond;
int number = 0;

void *producer(void *arg)
{
    for (int i = 0; i < 5; i++)
    {
        pthread_mutex_lock(&mutex);
        number++;
        printf("producer, number = %d\n", number);
        pthread_cond_signal(&cond);
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
}

void *consumer(void *arg)
{
    while (1)
    {
        pthread_mutex_lock(&mutex);
        if (number <= 0)
        {
            pthread_cond_wait(&cond, &mutex);
        }
        number--;
        printf("consumer1, number = %d\n", number);
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
}
```

```
void *consumer2(void *arg)
{
    while (1)
    {
        pthread_mutex_lock(&mutex);
        if (number <= 0)
        {
            pthread_cond_wait(&cond, &mutex);
        }
        number--;
        printf("consumer2, number = %d\n", number);
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
}



int main()
{
    pthread_mutex_init(&mutex, NULL);
    pthread_t tid1, tid2, tid3, tid4;
    pthread_create(&tid2, NULL, consumer, NULL);
    pthread_create(&tid3, NULL, consumer2, NULL);
    sleep(2);
    pthread_create(&tid1, NULL, producer, NULL);


    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
    pthread_join(tid3, NULL);
}
```

The result:

```
producer, number = 1
consumer1, number = 0
producer, number = 1
consumer2, number = 0
producer, number = 1
```

```
consumer2, number = 0
consumer1, number = -1
producer, number = 0
producer, number = 1
consumer2, number = 0
```

We can see that because if we use an if statement, when the threads wake up, they will not check the condition again. Therefore, they will continue executing the logic in the thread.

Why do both 2 consumer threads wake up at the same time even though there is only one signal from the producer thread in this line:

```
consumer2, number = 0
consumer1, number = -1
```

> According to man7 https://man7.org/linux/man-pages/man3/pthread_cond_wait.3p.html
>
> If a POSIX signal is delivered to a thread waiting on a condition variable, upon return from the signal handler, the thread may return zero from the wait function due to a spurious wakeup instead of resuming its sleep

As a result, we should follow the standard implement when we use variable condition

The thread synchronizations above are the methods that we want only 1 thread use the shared data. What about if we want more than 1 thread can use shared data at same time? Let's go to other synchronization: **semaphore**

Semaphore

A semaphore is a mechanism that allows us to limit the number of threads that can access a shared resource.

> Why do we need semaphore?

In a real device, resources such as CPU or RAM are not infinity. For example, if too many clients send requests to a host, the host may not be able to process all of them efficiently. As a result, the device can become slow, hot, and unable to maintain good performance.

> A binary semaphore is a semaphore whose counter can only have a value of 0 or 1. So, what is the difference between a binary semaphore and a mutex?

- With a mutex, the thread that locks it must unlock it. If that thread dies, other threads may be blocked forever. With a binary semaphore, any thread can signal it, and a timeout can help prevent indefinite blocking.
- When we use binary semaphore, we can know which threads can decrease the semaphore counter, while using mutex we can't

Bariers

Barriers are a synchronization mechanism that can be used to coordinate multiple threads working in parallel.

For example:

```
pthread_barrier_t barrier;

void *update_stock(void *arg)
{
    sleep(2);
    printf("thread stock\n");
    pthread_barrier_wait(&barrier);
    printf("Success");
    return NULL;
}

void *update_revenue(void *arg)
{
    sleep(3);
    printf("thread revenue\n");
    pthread_barrier_wait(&barrier);
}

void *update_customer_points(void *arg)
{
    sleep(1);
    printf("thread customer point\n");
    pthread_barrier_wait(&barrier);
}
```

```
void *create_invoice(void *arg)
{
    sleep(4);
    printf("thread invoice\n");
    pthread_barrier_wait(&barrier);
}

int main()
{
    int num_of_thread = 4;
    pthread_t threads[num_of_thread];

    pthread_barrier_init(&barrier, NULL, num_of_thread + 1);

    pthread_create(&threads[0], NULL, update_stock, NULL);
    pthread_create(&threads[1], NULL, update_revenue, NULL);
    pthread_create(&threads[2], NULL, update_customer_points, NULL);
    pthread_create(&threads[3], NULL, create_invoice, NULL);

    pthread_barrier_wait(&barrier);

    pthread_barrier_destroy(&barrier);

    return 0;
}
```

The result is:

```
thread customer point
thread stock
thread revenue
thread invoice
Success
```

In this example, the barrier will all thread put to sleep until all 5 threads done without using `phthread_join`

> If we call the init function twice, the number of counting threads will be overwritten by the last init function.
>
> If we destroy the barrier before using it, this action is undefined behavior. The program may not crash, but we cannot know the result of this action.

Spinlock

A spin lock is like a mutex, except that instead of blocking a process by sleeping, the process is blocked by busy-waiting (spinning) until the lock can be acquired.**Therefore, a spin lock wastes CPU resources**.

The question is, why don't we use a mutex instead? This method can also block the thread without wasting CPU resources.

To answer this question, let’s look at an example that compares the total runtime of a program using a spinlock and a mutex.

Source example: https://github.com/220222112-PhamNgocKhanh/Zhone_trainee/blob/main/thread_and_Synchronization/demo_spinlock_mutex.c

In this example, I use both a spinlock and a mutex to make one thread wait for another thread for a short and a long time and swapping 1000 times. The result when I run this program are:

```
===== SHORT CRITICAL SECTION =====
Spinlock: 1.051 ms
Mutex   : 2.414 ms

===== LONG CRITICAL SECTION =====
Spinlock: 2172.757 ms
Mutex   : 2029.748 ms
```

- In short time, the spinlock latency is better than the mutex because a spinlock does not need to depend on the scheduler. Therefore, the waiting thread can acquire the lock immediately after the other thread releases it.
- In long time, the results are quite similar because, over a long period, the time it takes for a thread to acquire the lock does not differ significantly. Moreover, sometimes the spinning thread wastes CPU resources, preventing other threads from using the CPU to run.

>

> We should use the spinlock when we know a thread will keep the lock in the short time

With a CPU 1 core, if we use spinlock, the program will really slow because 1 logical core for spinning thread, 1 for thread that keep the lock, other threads can't run.

Now we known some thread synchronization methods. However, what about if we want to make a function which use global or static variable become to a function that many threads can use without using mutex in all position calling of this function.

Reentrant and thread-specific function

Thread-safe function

- A function is said to be thread-safe if it can safely be invoked by multiple threads at the same time
- If a function is non thread-safe, if we call this function from other threads in executing time, race condition can occur

For example, this function is not thread-safe because if multiple threads invoke this function concurrently, the final value in glob is unpredictable

```
int glob = 0;

void func()
{
    glob++;
    printf("glob = %d\n", glob);
}
```

> We can easily make this function to safe by using synchronzation method like mutex.

Reentrant and non reentrant function

A reentrant function achieves thread safety without the use of mutexes. It does this by avoiding the use of global and static variables

> For several of the functions that have nonreentrant interfaces, SUSv3 specifies reentrant equivalents with names ending with the suffix _r like `strtok_r`, `rand_r`, `strerror_r`,...

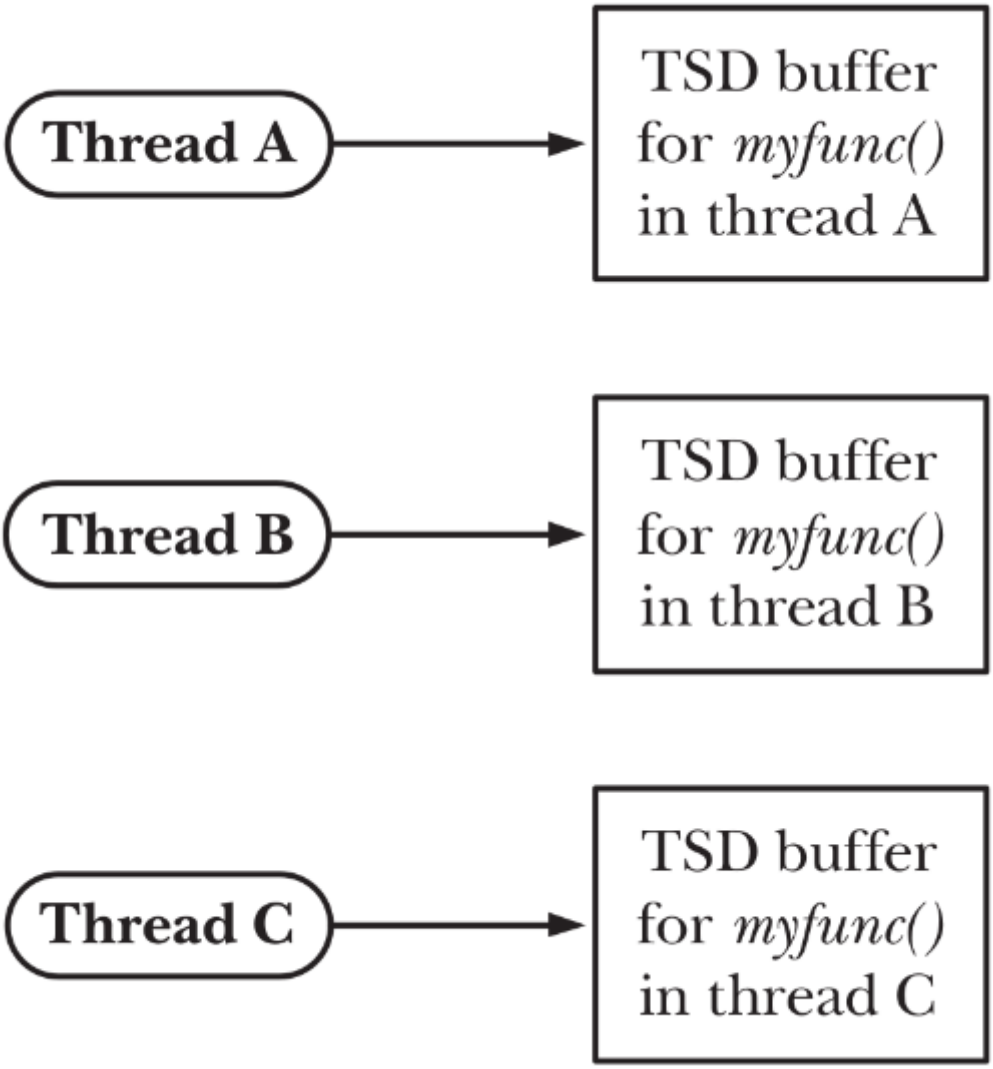
However, **reentrant function is not mean thread-safe function**, like this function, althought this function doesn't have any global or static variable, but if more than 1 thread call it at same time, **race condtion also occur**:

```
void increment(int *counter)
{
    (*counter)++;
}
```

Thread-specific data

Thread-specific data is a technique for making an existing function thread-safe without changing its interface.

Thread-specific data allows a function to maintain a separate copy of a variable for each thread that calls the function:



Now go to the example, I will build the `strtok` function:

```
char *state;

char *my_strtok(char *str, const char *delim)
{
    if (str != NULL)
        state = str;

    if (state == NULL)
        return NULL;

    while (*state != '\0' && strchr(delim, *state) != NULL)
        state++;

    if (*state == '\0')
```

```
        return NULL;

    char *start = state;

    while (*state != '\0' && strchr(delim, *state) == NULL)
        state++;

    if (*state != '\0')
    {
        *state = '\0';
        state++;
    }

    return start;
}

void *worker1(void *arg)
{
    char str[] = "A B C";

    printf("T1: %s\n", my_strtok(str, " "));

    printf("T1: %s\n", my_strtok(NULL, " "));
    printf("T1: %s\n", my_strtok(NULL, " "));

    return NULL;
}

void *worker2(void *arg)
{
    char str[] = "X Y Z";

    printf("T2: %s\n", my_strtok(str, " "));
    printf("T2: %s\n", my_strtok(NULL, " "));
    printf("T2: %s\n", my_strtok(NULL, " "));

    return NULL;
}

int main()
```

```
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, worker1, NULL);
    pthread_create(&t2, NULL, worker2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

The result:

```
T1: A
T1: Y
T1: Z
T2: X
T2: (null)
T2: (null)
```

Now we use the thread-specific technique to fix. We keep the source of 2 threads function.

```
pthread_key_t state_key;

char *my_strtok(char *str, const char *delim)
{
    char **state = pthread_getspecific(state_key);

    if (state == NULL)
    {
        state = malloc(sizeof(char *));
        *state = NULL;
        pthread_setspecific(state_key, state);
    }

    if (str != NULL)
        *state = str;

    if (*state == NULL)
```

```
        return NULL;

    while (**state != '\0' && strchr(delim, **state) != NULL)
        (*state)++;

    if (**state == '\0')
        return NULL;

    char *start = *state;

    while (**state != '\0' && strchr(delim, **state) == NULL)
        (*state)++;

    if (**state != '\0')
    {
        **state = '\0';
        (*state)++;
    }

    return start;
}

void free_state(void *ptr)
{
    free(ptr);
}

int main()
{
    pthread_t t1, t2;

    pthread_key_create(&state_key, free_state);

    pthread_create(&t1, NULL, worker1, NULL);
    pthread_create(&t2, NULL, worker2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_key_delete(state_key);
```

```
    return 0;
}
```

The result:

```
T1: A
T1: B
T1: C
T2: X
T2: Y
T2: Z
```

> We need to use thread-specific data when we want to maintain the state of a variable across multiple function calls within a thread. In a real project, this method will also keep the interface of a function too.

Sumary of synchronization

Situation	Method
Shared data can be accessed by only **one thread at a time** (the thread goes to sleep if it is locked).	**Mutex**
Limit the number of concurrent threads using shared data or managing a pool of identical resources.	**Semaphore (Counting)**
Only **one thread can modify** shared data at a time, but multiple threads can read it simultaneously.	**Read-Write Lock (RW Lock)**
Only one thread can access shared data at a time, but the critical section is extremely short; threads busy-wait/spin instead of sleeping to avoid context-switch overhead.	**Spinlock**
Multiple threads perform concurrent tasks and must wait for each other at a specific checkpoint. None can proceed until **all threads reach the checkpoint**.	**Barrier**
A thread needs to sleep and wait for a specific logical condition to become true (e.g., "queue is not empty") and be notified by another thread to wake up.	**Condition Variable (always used with a Mutex)**
A variable needs to be accessed globally across a thread's functions, but each thread must have its own independent, isolated copy, eliminating the need for locks.	**Thread-Specific Data**

Situation	Method
Strict execution ordering between threads (e.g., Thread B must wait for Thread A to finish a specific step before starting).	**Semaphore (Binary) / Condition Variable**

Thread and signal, fork and I/O

Thread and signal

When we send a signal to process, kernel will choose a thread of process to receive signal. We can aslo send signal to specific thread by `pthread_kill` function.

Each thread has thread mask, it help thread can avoid specific signal.

For example:

```
void signal_handler(int sig)
{
    printf("Thread %lu: received SIGUSR1\n",(unsigned long)pthread_self());
}

void *worker(void *arg)
{
    printf("Worker %d: started, tid = %lu\n",*(int *)arg,(unsigned long)pthread_self());

    while (1)
    {
        }

    return NULL;
}

int main(void)
{
    pthread_t t1, t2;
    int id1 = 1;
    int id2 = 2;
    pid_t pid = getpid();
```

```
    signal(SIGUSR1, signal_handler);

    pthread_create(&t1, NULL, worker, &id1);
    pthread_create(&t2, NULL, worker, &id2);

    kill(pid, SIGUSR1);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

The result:

```
Worker 1: started, tid = 136933367871040
Thread 136933371062080: received SIGUSR1
Worker 2: started, tid = 136933359478336
```

> In the result above, the main thread receives the signal. We can't know which thread main, worker1, or worker2 will receive the signal.

> We can use the `pthread_sigmask` function to block specific signals.

Thread and fork

As we know, `fork` is the function to create a child process. However, in multithread programming, when we call `fork` function in a thread, the child process has only this thread, not all threads of parent.

```
void *thread1(void *arg)
{
    pid_t p;
    p = fork();
    if (p == 0)
    {
```

```
        while (1)
        {
            int pid = getpid();
            printf("thread1 in Process: %d\n", pid);
            sleep(1);
        }
    }
    else
    {
        while (1)
        {
            int pid = getpid();
            printf("thread1 in Process: %d\n", pid);
            sleep(1);
        }
    }
}

int main(void)
{
    pthread_t t1;
    int pid = getpid();

    pthread_create(&t1, NULL, thread1, NULL);
    while (1)
    {
        printf("main thread in process: %d\n", pid);
        sleep(1);
    }

    pthread_join(t1, NULL);
}
```

The result:

```
main thread in process: 13853
thread1 in Process: 13853
thread1 in Process: 13855
main thread in process: 13853
thread1 in Process: 13853
```

```
thread1 in Process: 13855
main thread in process: 13853
thread1 in Process: 13853
thread1 in Process: 13855
thread1 in Process: 13855
main thread in process: 13853
thread1 in Process: 13853
main thread in process: 13853
thread1 in Process: 13855
thread1 in Process: 13853
thread1 in Process: 13855
thread1 in Process: 13853
```

We can see, the process with `pid = 13855` has only 1 thread.

> If the threads in the parent process hold any locks, the same locks will also be held in the child process. The problem is that the child process doesn't contain copies of the threads holding the locks, so there is no way for the child to know which locks are held and need to be unlocked. There for, if the child process lock mutex, this make **deadlock**

To fix this problem, we can use two approaches:

- Call the `exec` function to replace the current process and clear all lock states.
- Use the `pthread_atfork` function to check the lock state before forking, then unlock the lock in both the parent and child processes.

Threads and I/O

If two threads share the same file descriptor (fd) and both call `lseek` and `read` to read from different offsets in the file, a race condition can occur. We can avoid this by using `pread`, which atomically sets the offset and reads the data.

For example, we can replace this:

```
/* Thread 1 */
lseek(fd, 300, SEEK_SET);
read(fd, buf1, 100);

/* Thread 2 */
```

```
lseek(fd, 700, SEEK_SET);
read(fd, buf1, 100);
```

by

```
/* Thread 1 */
pread(fd, buf1, 100, 300);

/* Thread 2 */
pread(fd, buf2, 100, 700);
```

After understanding about thread overview, now we go to the bug section. In this section, we know the reason why the program run slow or never complete.

Stavation, livelock and deadlock

Before going to deadlock, we can see 2 bugs when we use mutex that we can see in real program: Starvation and livelock

Starvation

Stavation is the situation that 2 threads need a mutex lock to run. But thread 1 lock the mutex, run and unlock. However, after then, this thread will lock the mutex again, so on. Therefore, thread 2 need long time to take the mutex lock.

```
void *thread_A(void *arg)
{
    while (1)
    {
        pthread_mutex_lock(&lock);
        printf("A\n");
        pthread_mutex_unlock(&lock);
    }
}

void *thread_B(void *arg)
{
    while (1)
    {
```

```
        pthread_mutex_lock(&lock);
        printf("B\n");
        pthread_mutex_unlock(&lock);
    }
}
```

The result can be:

```
A
A
A
A
A
A
A
A
A
A
A
A
B
B
```

Livelock

Livelock is the bug when threads need more than 1 mutex lock to run. But they only take 1 lock, so they release this lock to avoid deadlock. However, other threads also do that, so the program can delay in long time without doing any thing.

```
void *thread_A(void *arg)
{
    while (1)
    {
        pthread_mutex_lock(&lock1);

        printf("A: got lock1\n");

        usleep(1000);

        if (pthread_mutex_trylock(&lock2) != 0)
```

```
        {
            printf("A: lock2 busy -> release lock1 and retry\n");

            pthread_mutex_unlock(&lock1);
            usleep(1000);
            continue;
        }

        printf("A: GOT BOTH LOCKS! Doing work...\n");

        pthread_mutex_unlock(&lock2);
        pthread_mutex_unlock(&lock1);

        break;
    }

    return NULL;
}

void *thread_B(void *arg)
{
    while (1)
    {
        pthread_mutex_lock(&lock2);

        printf("B: got lock2\n");

        usleep(1000);

        if (pthread_mutex_trylock(&lock1) != 0)
        {
            printf("B: lock1 busy -> release lock2 and retry\n");

            pthread_mutex_unlock(&lock2);
            usleep(1000);
            continue;
        }

        printf("B: GOT BOTH LOCKS! Doing work...\n");
```

```
        pthread_mutex_unlock(&lock1);
        pthread_mutex_unlock(&lock2);

        break;
    }

    return NULL;
}
```

The result when run 2 threads can like this:

```
A: got lock1
B: got lock2
B: lock1 busy -> release lock2 and retry
A: lock2 busy -> release lock1 and retry
B: got lock2
A: got lock1
A: lock2 busy -> release lock1 and retry
B: GOT BOTH LOCKS! Doing work...
A: got lock1
A: GOT BOTH LOCKS! Doing work...
```

Deadlock

Deadlock concept

A deadlock in multithreading is a state where two or more threads are stuck forever because each thread waits for a resource or lock held by another.

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void *thread_A(void *arg)
{
    while (1)
    {
        pthread_mutex_lock(&lock1);
        sleep(3);
        pthread_mutex_lock(&lock2);
```

```
        printf("A: GOT BOTH LOCKS! Doing work...\n");

        pthread_mutex_unlock(&lock2);
        pthread_mutex_unlock(&lock1);

        break;
    }

    return NULL;
}

void *thread_B(void *arg)
{
    while (1)
    {
        pthread_mutex_lock(&lock2);
        pthread_mutex_lock(&lock1);


        printf("B: GOT BOTH LOCKS! Doing work...\n");

        pthread_mutex_unlock(&lock1);
        pthread_mutex_unlock(&lock2);

        break;
    }

    return NULL;
}

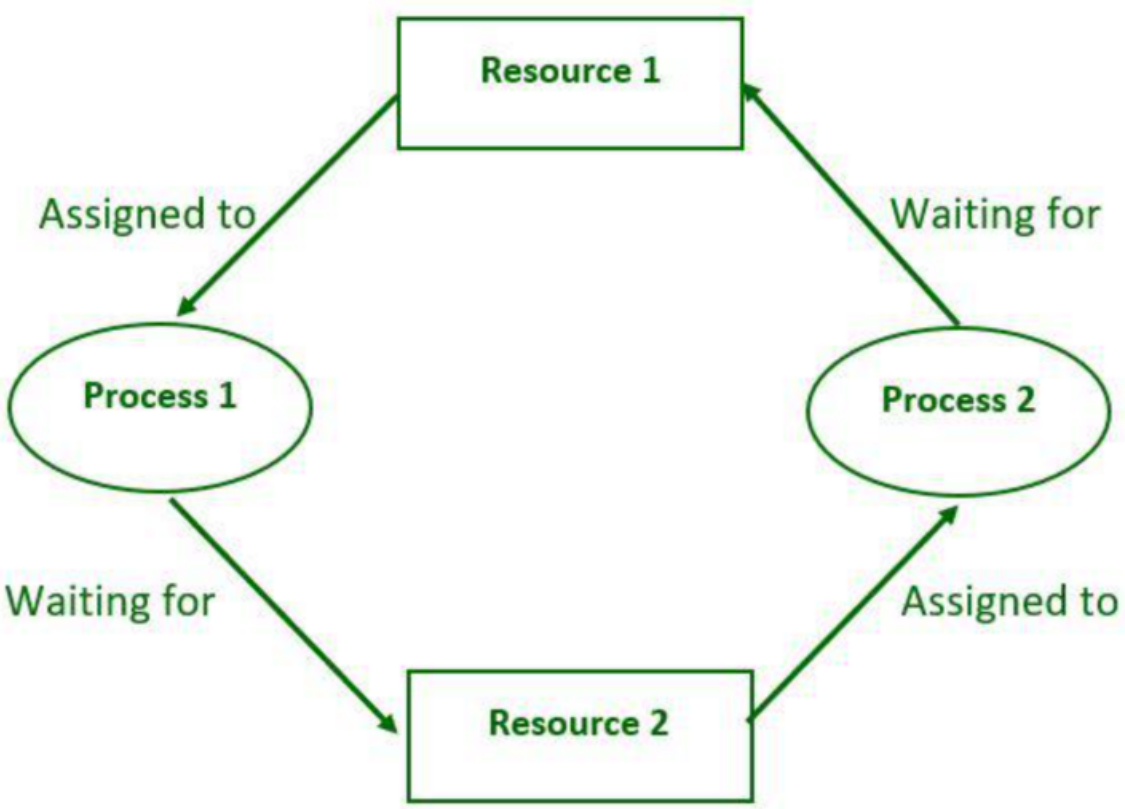
int main()
{
    pthread_t A, B;

    pthread_create(&A, NULL, thread_A, NULL);
    sleep(1);
    pthread_create(&B, NULL, thread_B, NULL);

    pthread_join(A, NULL);
```

```
    pthread_join(B, NULL);

    return 0;
}
```



When we run the program above, the deadlock will occur. Now we need to know the conditions for deadlock

Deadlock conditions

There are 4 conditions for deadlock:

- **Mutual Exclusion**: Only one thread can use a resource at a time. (Example: a pthread_mutex_t locked by Thread A).
- **Hold and Wait**: A thread holds at least one resource while waiting to get another resource held by a different thread.
- **No Preemption**: The operating system cannot forcibly take a resource away from a thread; the thread must release it voluntarily.
- **Circular Wait**: A closed loop of threads exists. Thread 1 waits for Thread 2's resource, and Thread 2 waits for Thread 1's resource

Deadlock Prevention

There are several ways to prevent deadlocks:

- **All-or-nothing**: In this method, a thread tries to acquire all the mutexes it needs. If it cannot acquire one of them, it releases all the mutexes it currently holds and waits for a short time before trying again. This method prevents the hold-and-wait condition. However, it may cause livelock and reduce system performance because threads may repeatedly release and reacquire locks.
- **Lock ordering**: In this method, all locks in the system are assigned a specific order or priority. When a thread needs to acquire multiple locks, it must always acquire them in the same order. For example, if Lock A has a higher priority than Lock B, the thread must always acquire Lock A before Lock B. This prevents circular wait, which is one of the necessary conditions for deadlock.
- **Global lock**: Instead of using multiple locks for different critical sections or shared resources, we can use a single global lock to protect all shared data. This approach can simplify synchronization and prevent deadlocks caused by multiple locks. However, it reduces concurrency and flexibility because only one thread can access the protected data at a time.
- **Try-lock and timed-lock**: Functions such as pthread_mutex_trylock() and pthread_mutex_timedlock() can be used to avoid waiting indefinitely for a mutex. pthread_mutex_trylock() returns immediately if the mutex is unavailable, while pthread_mutex_timedlock() waits only until a specified timeout. These functions can help a program detect or handle situations that could otherwise lead to indefinite waiting.

Deadlock Detection

To detect and investigate deadlocks, we can use debugging and analysis tools such as **GDB** and **Valgrind**. With `GDB`, we can inspect the state of each thread and determine which mutex a thread is holding and which mutex it is waiting for. `Valgrind`, especially tools such as `Helgrind`, can help detect potential problems related to thread synchronization, including lock-order violations and possible deadlocks.

Atocmic instruction and futex

Atomic instruction

As we know in the previous part, we know the race condition situation. What is read race condition explain in hardware layer.

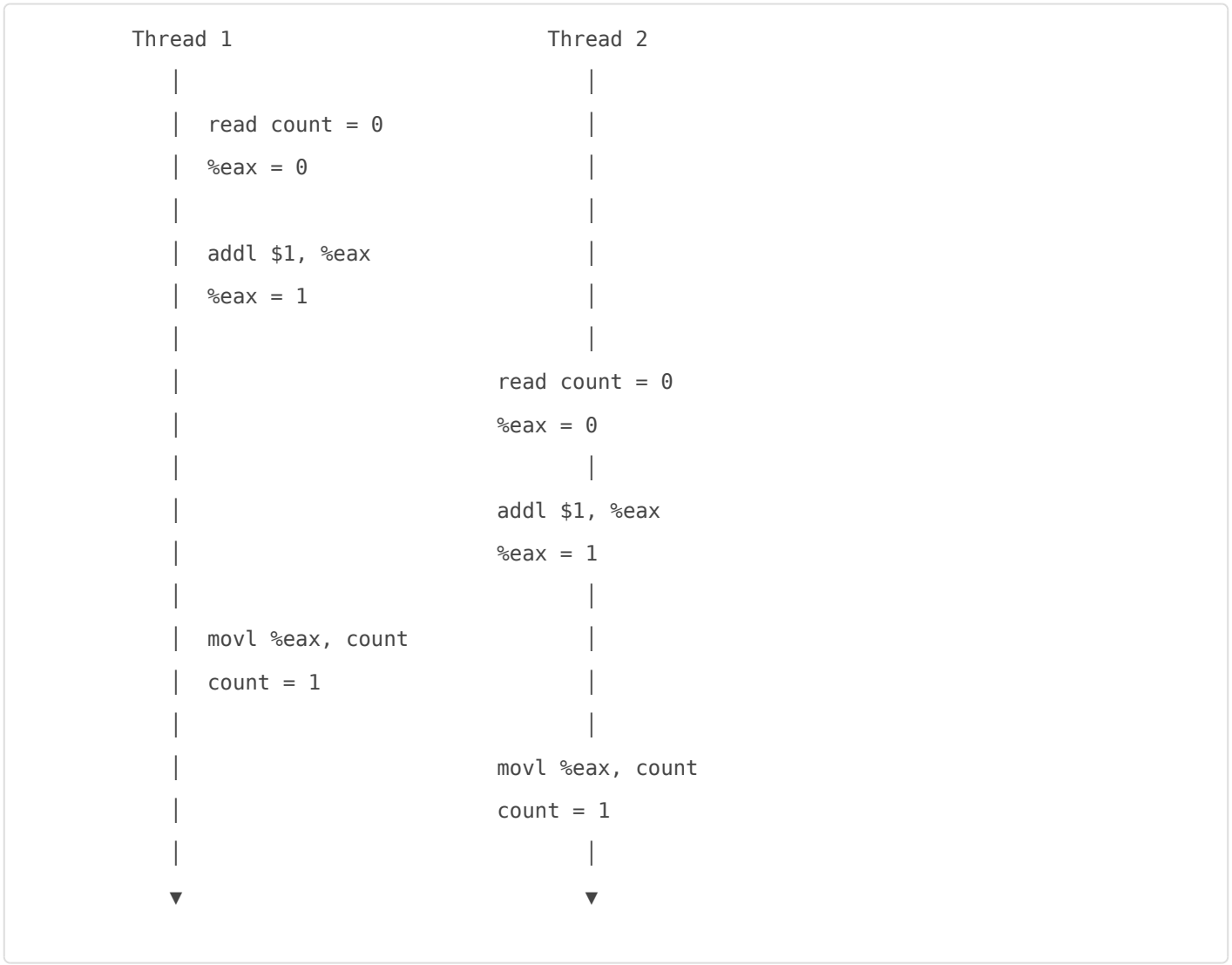
To explain:

```
count++;
```

This C line will be presented in assembly code like this:

```
addl $1, %eax
movl %eax, count(%rip)
movl $0, %eax
```

So when 2 threads run this instuctions at the same time, we can see an situation of race condition bellow:



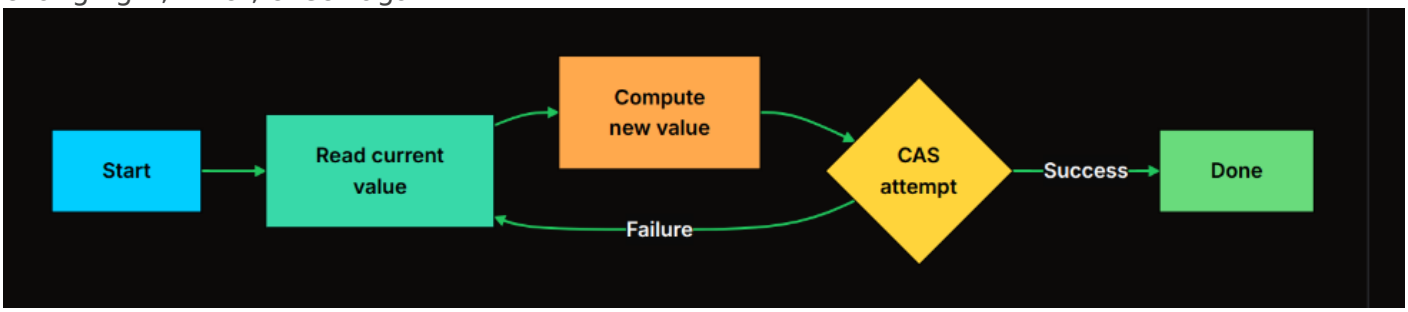
x86-64 offers a set of instructions to solve this problem, by making an instruction indivisible. It behaves as a single unit not only for the core that processes it, but for every other core as well. An operation that holds together, that no other core can split, is called **atomic**.

In x86 instruction provide a instuction: Compare and swap(CAS) to solve this situation.

CAS is an atomic operation that takes three arguments: a memory location, an expected value, and a new value. It atomically compares the current value at the memory location with the expected value. If they match, it replaces the current value with the new value and returns success. If they do not match, it leaves the memory unchanged and returns failure.

```
function CAS(location, expected, new):
    atomic:
        if *location == expected:
            *location = new
            return true
        else:
            return false
```

So we can use CAS to check the value of variable before changing, if the value is same expected, changing it, if not, check again:



This is the way spinlock work. Now try to deeper information, we will understand how CAS instruction do in hardware layer.

Cache conherence problem

CPU has cache, this memory will temporary store the value from memory. We need cache because it help cpu read the data faster than read from ram. Each CPU core will have separate cache.

Cache conherence is the problem when more than 1 threads read cache memory for the same location at same time. For example:

Time step	Event	Cache content for CPU A	Cache content for CPU B	Memory contents for location X
0				0
1	CPU A read X	0		0
2	CPU B read X	0	0	0
3	CPU A store 1 into X	1	0	1

So we can see although the value of X changed in memory to 1, but the value in cache for CPU B still is 0.

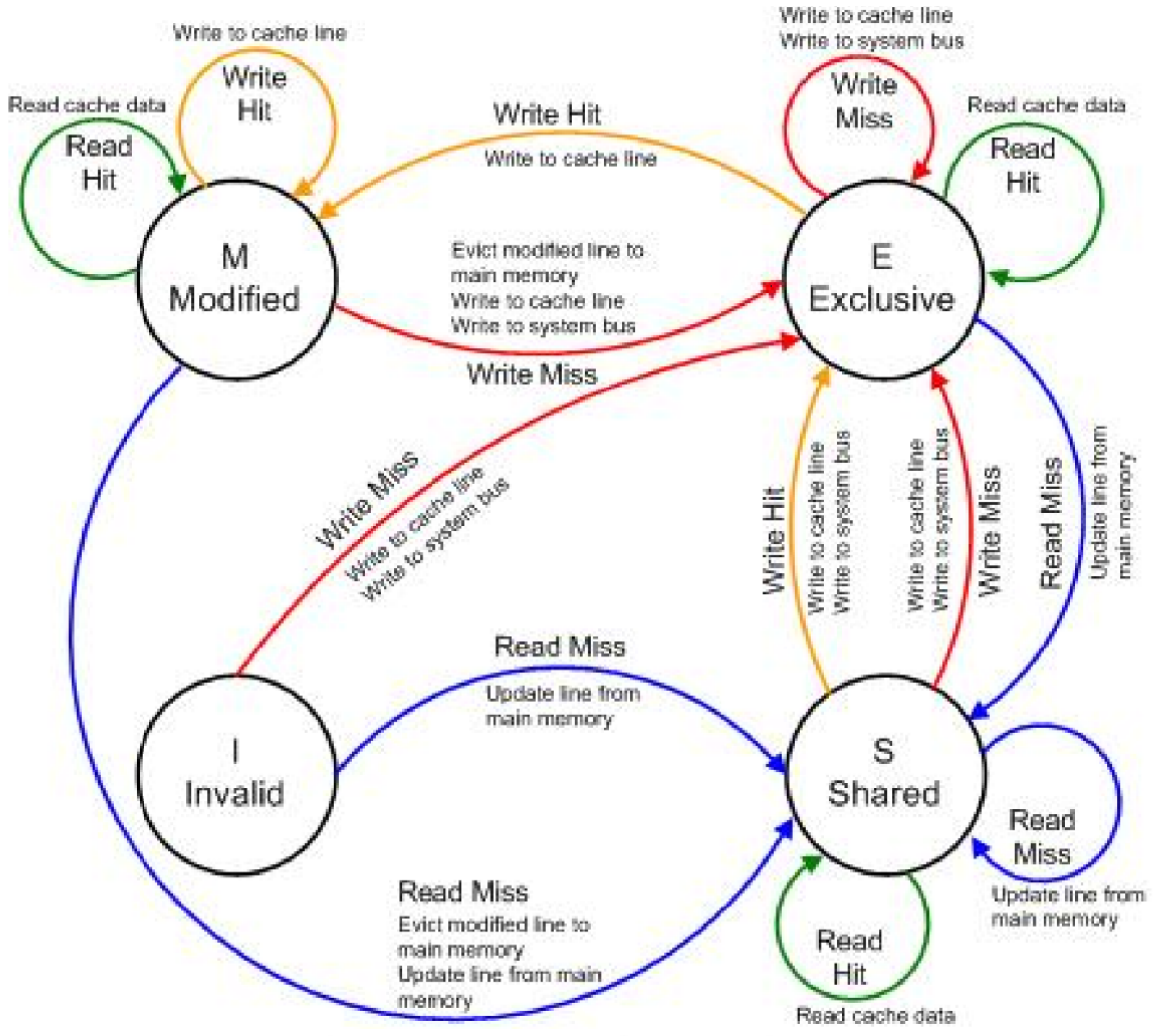
> How CPU solve this problem?

One method of enforcing coherence is to ensure that a processor has exclusive access to a data item before it writes that item. This style of protocol is called a **write invalidate protocol** because it invalidates copies in other caches on a write.

Exclusive access ensures that no other readable or writable copies of an item exist when the write occurs: all other cached copies of the item are invalidated.

Follow by MESI protocol, each cache line has 4 stage:

- **Invalid**: It is a non-valid state. The data you are looking for are not in the cache, or the local copy of these data is not correct because another processor has updated the corresponding memory position.
- **Shared**: Shared without having been modified. Another processor can have the data into the cache memory and both copies are in their current version.
- **Exclusive**: Exclusive without having been modified. That is, this cache is the only one that has the correct value of the block. Data blocks are according to the existing ones in the main memory
- **Modified**: Actually, it is an exclusive-modified state. It means that the cache has the only copy that is correct in the whole system. The data which are in the main memory are wrong



If any CPU want to change the state, they need request to cache-coherence system, when this unit accept, the state of this CPU will change.

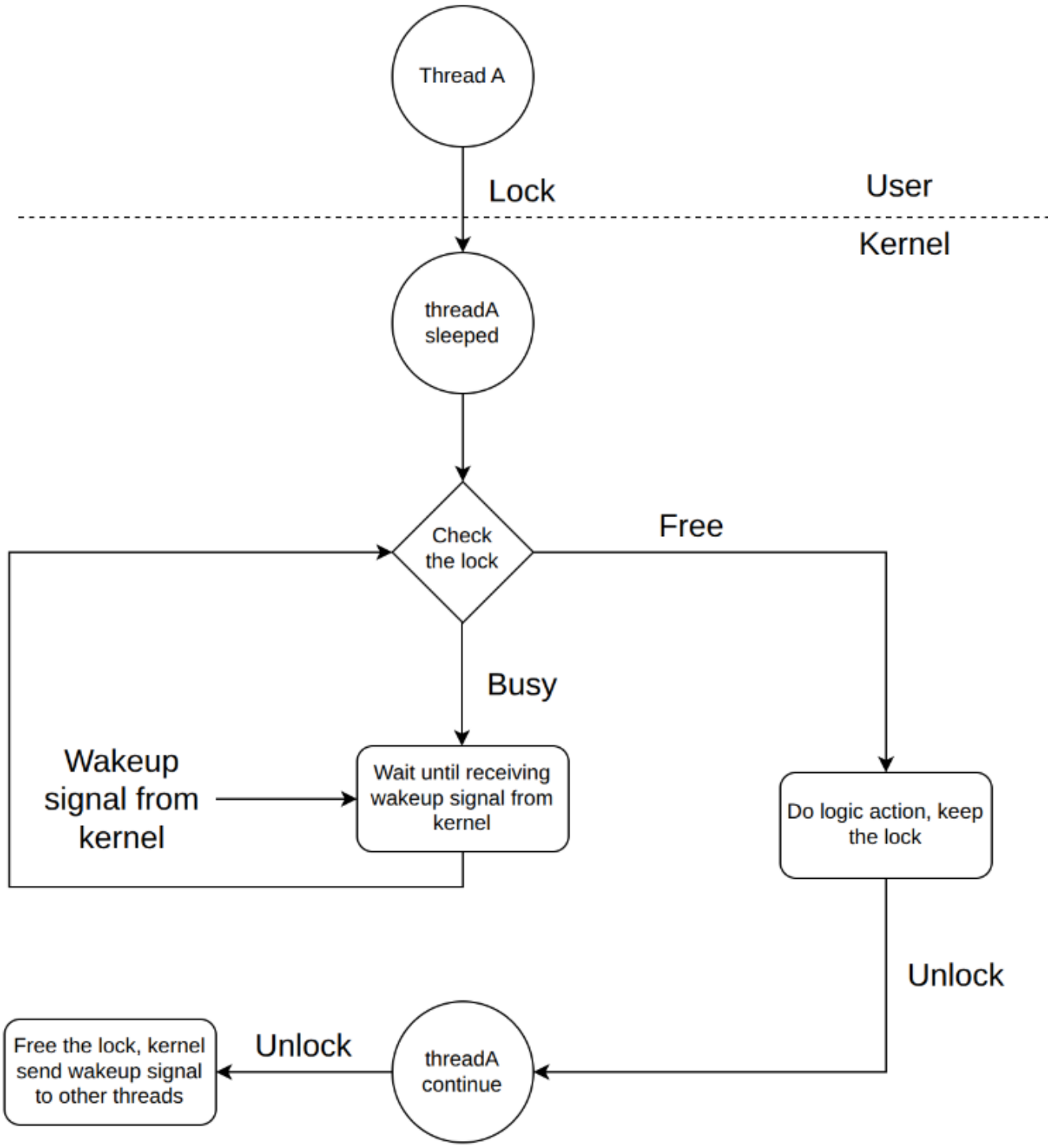
> Now we know the way spinlock is implemented, what about mutex?

Futex

As we know, system call function always slower than operation in userspace. If we want to make a thread sleep, only kernel can do that. By using futex, we can avoid the sleeping and calling to kernel in the situation when the lock is free.

For example, when the older mutex, because we don't check the status of this lock in userspace, so we need to call the lock system call to check in kernel space.

```
lock();
x++;
unlock();
```



A mutex can be implemented using only the kernel (which is expensive because it requires syscalls), or only in userspace (which is expensive since waiting requires spinning, and it is hard to implement things like queueing and fairness). Futexes provide a middle ground where the kernel implements the thing only the kernel can do: put threads to sleep and wake them up appropriately.

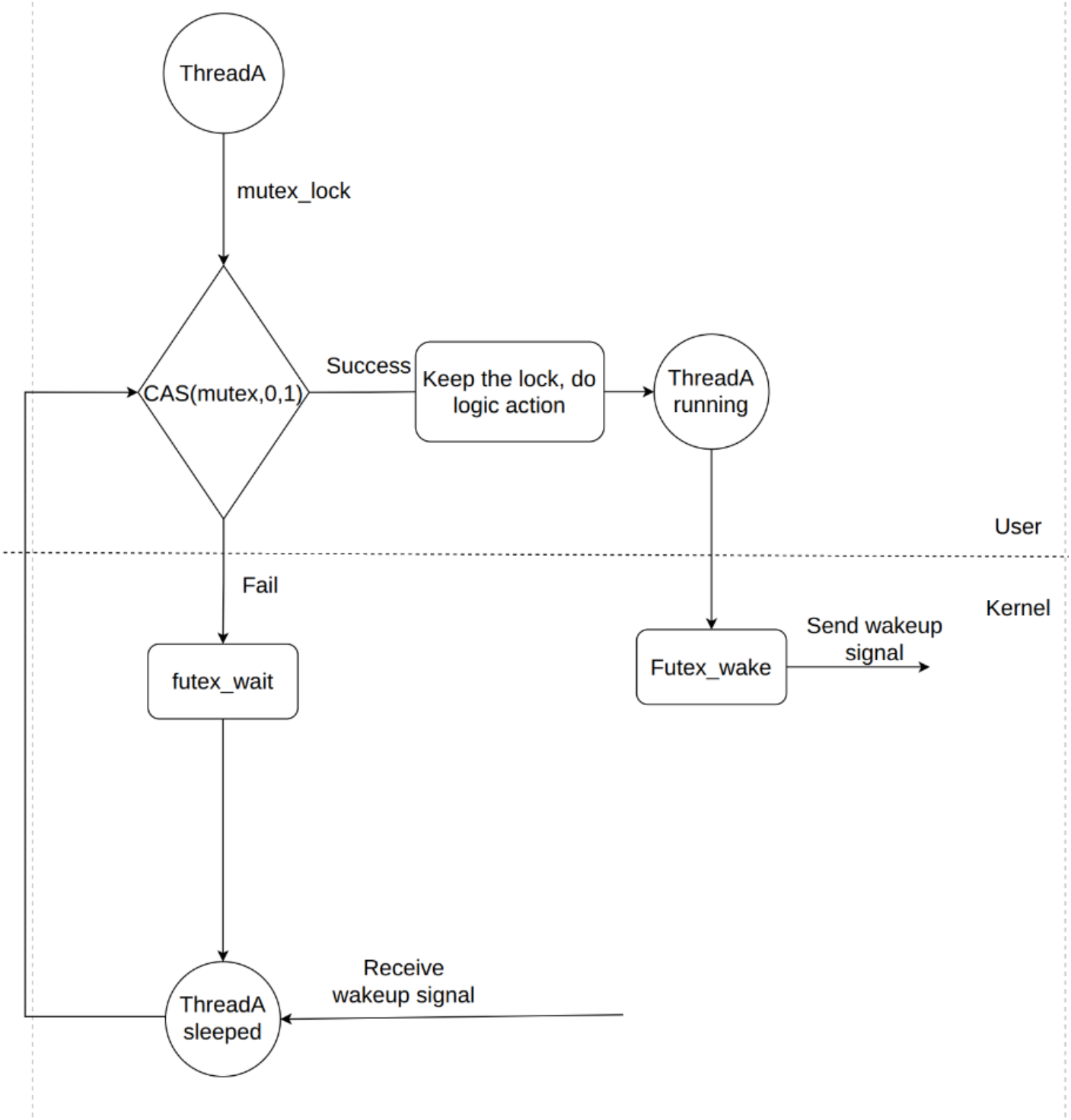
Futex has 2 important syscalls:

- futex_wake: Send a wakeup signal to another thread which still waiting the signal
- futex_wait: Wait until receive an signal from futex_wake

```
typedef mutex_t uint32_t;

void mutex_lock(mutex_t *m) {
  while (!atomic_cas(m, UNLOCKED, LOCKED)) {
    futex_wait(m, LOCKED);
  }
}

void mutex_unlock(mutex_t *m) {
  atomic_store(m, UNLOCKED);
  futex_wake(m, 1);
}
```



Labs practice

Lab1: Race condition https://github.com/220222112-PhamNgocKhanh/Zhone_trainee/tree/main/thread_and_Synchronization/lab1_race_condition

Lab2: Reentrant function https://github.com/220222112-PhamNgocKhanh/Zhone_trainee/tree/main/thread_and_Synchronization/lab2_reentrant_function

Lab 3: Producer-customer https://github.com/220222112-PhamNgocKhanh/Zhone_trainee/tree/main/thread_and_Synchronization/lab3_producer_customer

Lab 4: Deadlock https://github.com/220222112-PhamNgocKhanh/Zhone_trainee/tree/main/thread_and_Synchronization/lab4_deadlock

---

Revision #10
Created 2026-08-27 01:55:17 UTC by Khanh Pham
Updated 2026-09-08 02:47:49 UTC by Khanh Pham